



man what the heck is this there is nothing explained here from my code that i have given you break it block by block and then explain each function what it does hwo it does and why it does their time complexities not just simple like suppose ifits just simply say n derive how did it come o n

suppose a question for quicksort

$n / 2^0$ elems in level 0

$n / 2^2$ elems in level 2

similarly

$n / 2^x$ elems in x level

so

let $n / 2^x = 1$

$n = 2^x$

$\log n = \log 2^x$

$\log n = x \log 2(2)$

$x = \log n$

now we know that the stack will maximum will be pushed till x-2 levels so maximum level of stack will be x-1

so $sc = x - 1 = \log 2n - 1$ so $s.C = O(\log 2n)$

this is how i derived space complexity for quicksort algorihm

so similarly you have to derive for all the function that we are dealing with explain in detail their working explain each primitive call as well

This is a comprehensive, line-by-line breakdown of your specific code, with mathematical derivations for Time (TC) and Space Complexity (SC) derived from first principles as you requested.

I have broken this down into the three main classes in your file: **BTree**, **Heap**, and the **BST Helper Functions**.

Part 1: The `BTree` Class (Binary Tree Primitives)

This class handles the structural logic of a standard binary tree node.

1. `__init__(self, item)`

```
def __init__(self, item):
    self.data = item    # Assignment: O(1)
    self.lc = None      # Assignment: O(1)
    self.rc = None      # Assignment: O(1)
```

Explanation:

- **Primitive Call:** `self.data = item` stores the payload.
- **Primitive Call:** `self.lc = None` initializes the left pointer.
- **Complexity:**
 - **TC:** Simple assignment operations. $1 + 1 + 1 = 3$ steps. Constant time **O(1)**.
 - **SC:** Allocates memory for one object. **O(1)**.

2. `traversebf(self)` (Breadth-First Search)

This is a critical function. It prints nodes level-by-level.

```
def traversebf(self):
    nodes = [self]          # Initialize Queue with root
    print(self.data, end=" ")
    i = 0                   # Index of node being processed
    n = 1                    # Current count of nodes found
    while i < n:             # Loop until we processed all found nodes
        p = nodes[i]        # Access current node O(1)
        if p.lc is not None:
            print(p.lc.data, end=" ")
            nodes.append(p.lc) # Enqueue Left Child
            n += 1            # Increment count
        if p.rc is not None:
            print(p.rc.data, end=" ")
            nodes.append(p.rc) # Enqueue Right Child
            n += 1            # Increment count
        i += 1              # Move to next node
    print()
```

Detailed Logic:

- This uses a **list as a queue**.
- `i` points to the "head" of the queue (element being processed).
- `n` points to the total elements "enqueued" so far.
- `nodes.append(p.lc)` adds the child to the end of the list (effectively enqueueing it).

Derivation of Time Complexity:

Let N be the total number of nodes in the tree.

1. The `while` loop runs as long as $i < n$.
2. Every valid node in the tree is added to the `nodes` list exactly once (when its parent is processed).
3. Therefore, n will eventually reach N .
4. Inside the loop, we perform constant time checks (`if p.lc is not None`) and appends.
5. Total operations $\approx c \cdot N$ (where c is operations per node).
6. **TC = $O(N)$.**

Derivation of Space Complexity:

1. The list `nodes` stores references to **every node** in the tree by the time the function finishes.
2. In a standard BFS queue, we usually remove elements from the front. However, your code **never removes elements** (`i` just increments).
3. Therefore, the list grows to size N .
4. **SC = $O(N)$.** (Note: Standard BFS is $O(\text{width})$, but this specific implementation keeps all nodes in memory).

3. `height(self)`

Calculates the longest path from this node to a leaf.

```
def height(self):
    if self.rc is None and self.lc is None: return 1 # Base case: Leaf
    rh = 0
    lh = 0
    if self.rc is not None: rh = self.rc.height() # Recurse Right
    if self.lc is not None: lh = self.lc.height() # Recurse Left
    if rh > lh: return rh + 1 # Max + 1
    return lh + 1
```

Derivation of Time Complexity:

Let $T(N)$ be the time to calculate height for a tree of N nodes.

1. The function visits the current node (cost c_1).
2. It recursively visits the left child (size N_{left}) and right child (size N_{right}).
3. Recurrence relation: $T(N) = T(N_{left}) + T(N_{right}) + c_1$.
4. Since $N_{left} + N_{right} = N - 1$ (all nodes except root), this expands to:
 $T(N) = c_1 + c_1 + \dots + c_1$ (N times)
5. **TC = $O(N)$.**

Derivation of Space Complexity:

1. This depends on the Recursion Stack depth.

2. Best Case (Perfect Tree):

- Nodes split evenly: $N/2^x = 1 \implies 2^x = N \implies x = \log_2 N$.
- Stack depth = x . **SC = $O(\log N)$.**

3. Worst Case (Skewed Tree):

- Tree looks like a linked list (only left children).
- Depth = N . **SC = $O(N)$.**

4. isStrictlyBinary(self)

Checks if every node has either 0 or 2 children.

```
def isStrictlyBinary(self):
    if self.lc is None:
        if self.rc is None: return True # Case: Leaf (0 children) -> Valid
        return False # Case: No Left, Yes Right -> Invalid
    if self.rc is None: return False # Case: Yes Left, No Right -> Invalid
    return self.lc.isStrictlyBinary() and self.rc.isStrictlyBinary() # Recurse
```

Derivation:

1. **Logic:** If a node violates the condition (has 1 child), it returns False immediately. If it's valid, it checks children.
2. **Worst Case:** The tree is strictly binary (valid). The function must visit every node to confirm.
3. Summing work across all nodes: $\sum_{i=1}^N c = O(N)$.
4. **TC = $O(N)$.**

5. isPerfect(self)

Checks if all levels are filled. A perfect tree of height h has $2^h - 1$ nodes.

```
def isPerfect(self):
    h = self.height() # O(N) scan
    n = self.nodecount() # O(N) scan
    m = pow(2, h) - 1 # O(1) math
    if n == m: return True
    return False
```

Derivation:

1. Calls `self.height()`: We proved this is $O(N)$.
2. Calls `self.nodecount()`: This visits every node, so $O(N)$.
3. Total Time: $O(N) + O(N) = 2O(N) \approx O(N)$.
4. **TC = $O(N)$.**

Part 2: The heap Class (Max Heap)

This class implements a Priority Queue using a list.

1. rise(self) (Up-Heapify)

Moves the last element up to its correct position.

```
def rise(self):
    index = len(self) - 1      # Start at last leaf
    p = (index - 1) // 2      # Formula for parent index
    while p >= 0:              # Loop until root
        if self.hlist[index] <= self.hlist[p]: return # Heap Property met
        self.swap(index,p)     # Else, swap up
        index = p              # Move pointer up
        p = (index - 1) // 2   # Recalculate parent
```

Mathematical Derivation:

1. Let N be the number of nodes in the heap.
2. The heap is a Complete Binary Tree.
3. The logic moves from a leaf node up towards the root.
4. **Level Analysis:**
 - Level 0 (Root): 1 node
 - Level k : 2^k nodes.
 - Total nodes $N \approx 2^H$, where H is height.
 - Taking log of both sides: $\log_2 N = H$.
5. In the worst case (inserted element is the new maximum), we swap at every level from bottom to top.
6. Number of iterations = Height of tree = H .
7. Since $H = \log_2 N$:
8. **TC = $O(\log N)$.**

2. sink(self, lenheap) (Down-Heapify)

Moves the root element down to its correct position. Used after deletion.

```
def sink(self, lenheap):
    index = 0                # Start at root
    lc = 2 * index + 1        # Left child formula
    while lc < lenheap:       # While left child exists
        # ... comparisons to find largest child ...
        # ... swap with largest child ...
        # index = child_index    # Move pointer down
```

Mathematical Derivation:

1. Logic follows a path from the **Root** down to a **Leaf**.
2. At each step, the algorithm performs constant comparisons (compare left vs right child, compare parent vs largest child).
3. The maximum path length in a complete binary tree is the height H .
4. As derived in rise, $H = \log_2 N$.
5. **TC = $O(\log N)$.**

3. add_to_heap(self, item)

```
def add_to_heap(self, item):
    self.hlist.append(item) #  $O(1)$  amortized
    self.rise()             #  $O(\log N)$ 
```

TC = $O(\log N)$.

4. del_from_heap(self)

```
def del_from_heap(self):
    x = self.hlist[0]          # Save max
    self.hlist[0] = self.hlist.pop() # Move last element to root  $O(1)$ 
    self.sink(len(self))       # Fix the heap  $O(\log N)$ 
    return x
```

TC = $O(\log N)$.

Part 3: BST Functions (Binary Search Tree)

These functions operate on BTree nodes but assume the BST property (Left < Root < Right).

1. searchbst(node, target)

Finds a node or its potential parent.

```
def searchbst(node, target):
    if node is None: return [None, 0]
    if node.data == target: return [node, 1]
    if node.data > target:
        node = node.lc # Move Left
    else:
        node = node.rc # Move Right
    return searchbst(node, target)
```

Mathematical Derivation (Time Complexity):

1. This is a Decrease-and-Conquer algorithm.

2. At each step, we discard one subtree (left or right).

3. **Best/Average Case (Balanced Tree):**

- At step 0: Problem size N .
- At step 1: Problem size $N/2$.
- At step k : Problem size $N/2^k$.
- Stop when $N/2^k = 1 \implies k = \log_2 N$.
- **TC = $O(\log N)$.**

4. **Worst Case (Skewed Tree):**

- Tree is a straight line ($10 \rightarrow 9 \rightarrow 8 \rightarrow \dots$).
- At step 1: Problem size $N - 1$.
- We visit every node.
- **TC = $O(N)$.**

Space Complexity:

- The function calls itself recursively.
- Stack Depth = Height of tree (H).
- **SC = $O(H)$** (which is $O(\log N)$ balanced, $O(N)$ worst).

2. `addtobst(root, item)`

Inserts an item.

```
def addtobst(root, item):
    parent = root
    # ... Loop logic ...
    while added is False:
        if item < parent.data:
            parent = parent.lc # Move Left
        else:
            parent = parent.rc # Move Right
```

Derivation:

1. This is **Iterative**, not recursive.
2. The `while` loop traverses a single path from root to a null pointer (where the new node goes).
3. Max iterations = Height of tree (H).
4. **TC = $O(H)$** ($O(\log N)$ average, $O(N)$ worst).
5. **SC = $O(1)$** . (No recursion stack, just a `parent` pointer).

3. `delfrombst(root, item)`

Deletes a node and restructures.

```
def delfrombst(root, item):
    x = searchbst(root, item) # 1. Find the node: O(H)
    # ... case handling ...
    # If 2 children:
    y = minimum(root.rc)      # 2. Find successor: O(H)
    z = searchbst(root.rc, y) # 3. Find successor's parent: O(H)
    # Pointer updates: O(1)
```

Derivation:

1. **Step 1 (searchbst):** Costs $O(H)$.
2. **Step 2 (minimum):** Traverses strictly left from the right child. Max steps = H . Cost $O(H)$.
3. **Step 3 (searchbst again):** Searches for the successor y . Cost $O(H)$.
4. **Total Time:** $T(N) = c_1H + c_2H + c_3H \approx 3H$.
5. Since constants drop out in Big-O:
6. **TC = $O(H)$.**

Summary Table (for your exam)

Function	Time Complexity (Balanced)	Time Complexity (Worst)	Space Complexity	Reason
BTree				
<code>traversebtf</code>	$O(N)$	$O(N)$	$O(N)$	Enqueues every node.
<code>traversein</code>	$O(N)$	$O(N)$	$O(H)$	Visits all nodes; stack stores height.
<code>height</code>	$O(N)$	$O(N)$	$O(H)$	Must visit all to find max depth.
<code>isPerfect</code>	$O(N)$	$O(N)$	$O(H)$	Calls <code>height</code> and <code>nodecount</code> .
Heap				
<code>rise</code>	$O(\log N)$	$O(\log N)$	$O(1)$	Path is strictly root-to-leaf height.
<code>sink</code>	$O(\log N)$	$O(\log N)$	$O(1)$	Path is strictly root-to-leaf height.
BST				
<code>searchbst</code>	$O(\log N)$	$O(N)$	$O(H)$	Discards half tree (avg) or 1 node (worst).
<code>addtobst</code>	$O(\log N)$	$O(N)$	$O(1)$	Iterative path descent.

Function	Time Complexity (Balanced)	Time Complexity (Worst)	Space Complexity	Reason
delfrombst	$O(\log N)$	$O(N)$	$O(H)$	Search + Minimum + Pointer Swaps.